國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab09

姓名：林彥廷

學號：112370219

Lab01-Q1.
Delete Node in a Binary Search Tree
Code

```cpp
#include <iostream>
using namespace std;

// ========================
// 樹的節點類別
// ========================
class TreeNode {
public:
    int data;           // 節點儲存的資料
    TreeNode* left;     // 指向左子節點
    TreeNode* right;    // 指向右子節點

    // 節點建構子：設定資料並初始化左右指標為空
    TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}
};

// ========================
// 二元搜尋樹類別
// ========================
class BinarySearchTree {
private:
    TreeNode* root; // 根節點指標

    // --- 私有的遞迴插入方法 ---
    TreeNode* insert(TreeNode* node, int val) {
        if (node == nullptr) {
            return new TreeNode(val); // 找到插入位置，建立新節點
        }

        if (val < node->data) {
            node->left = insert(node->left, val); // 插入左子樹
        }
        else if (val > node->data) {
            node->right = insert(node->right, val); // 插入右子樹
```

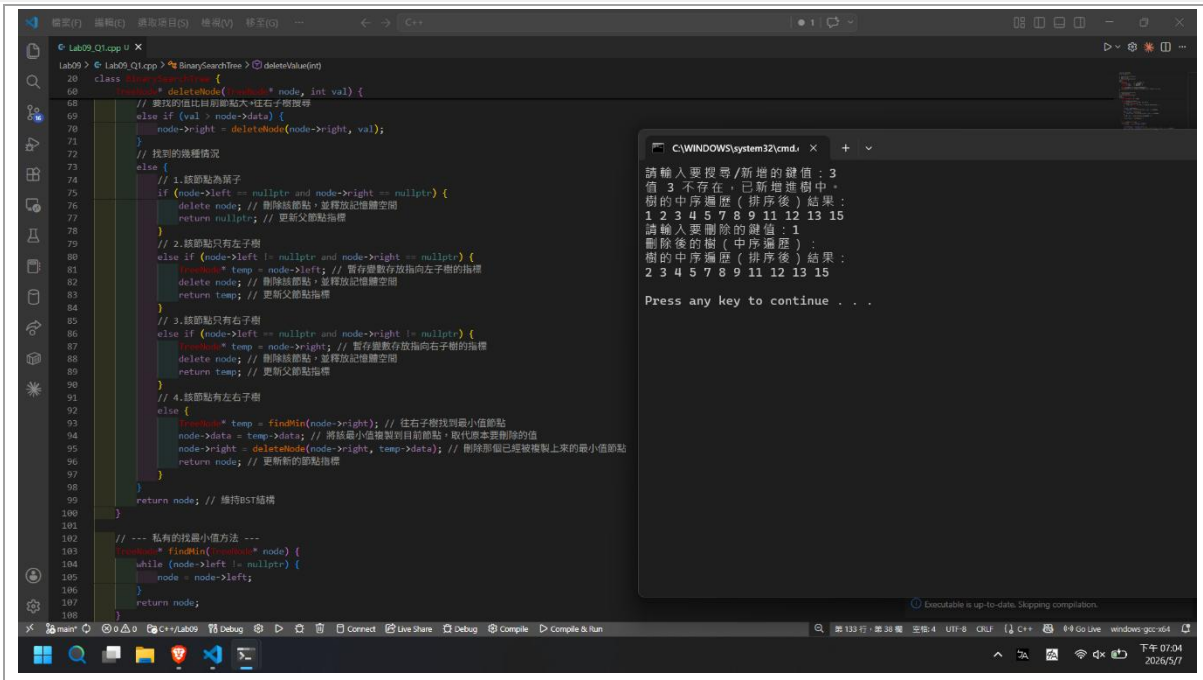
```
        }
        // 如果 val == node->data，不插入（避免重複）

        return node; // 回傳更新後的節點
    }

    // --- 私有的遞迴搜尋方法 ---
    bool search(TreeNode* node, int target) {
        if (node == nullptr) return false;

        if (node->data == target) return true;
        else if (target < node->data) return search(node->left, target); // 繼續
在左子樹找
        else return search(node->right, target);// 繼續在右子樹找
    }

    // --- 私有的中序遍歷方法（左-根-右）---
    void inOrderTraversal(TreeNode* node) {
        if (node == nullptr) return;

        inOrderTraversal(node->left);   // 先走訪左子樹
        cout << node->data << " ";   // 印出目前節點的資料
        inOrderTraversal(node->right); // 再走訪右子樹
    }

    // --- 私有的刪除方法 ---
    TreeNode* deleteNode(TreeNode* node, int val) {
        if (node == nullptr) return nullptr; // 找不到節點，就回傳 nullptr

        // 要找的值比目前節點小→往左子樹搜尋
        if (val < node->data) {
            node->left = deleteNode(node->left, val);
        }

        // 要找的值比目前節點大→往右子樹搜尋
        else if (val > node->data) {
            node->right = deleteNode(node->right, val);
        }
```

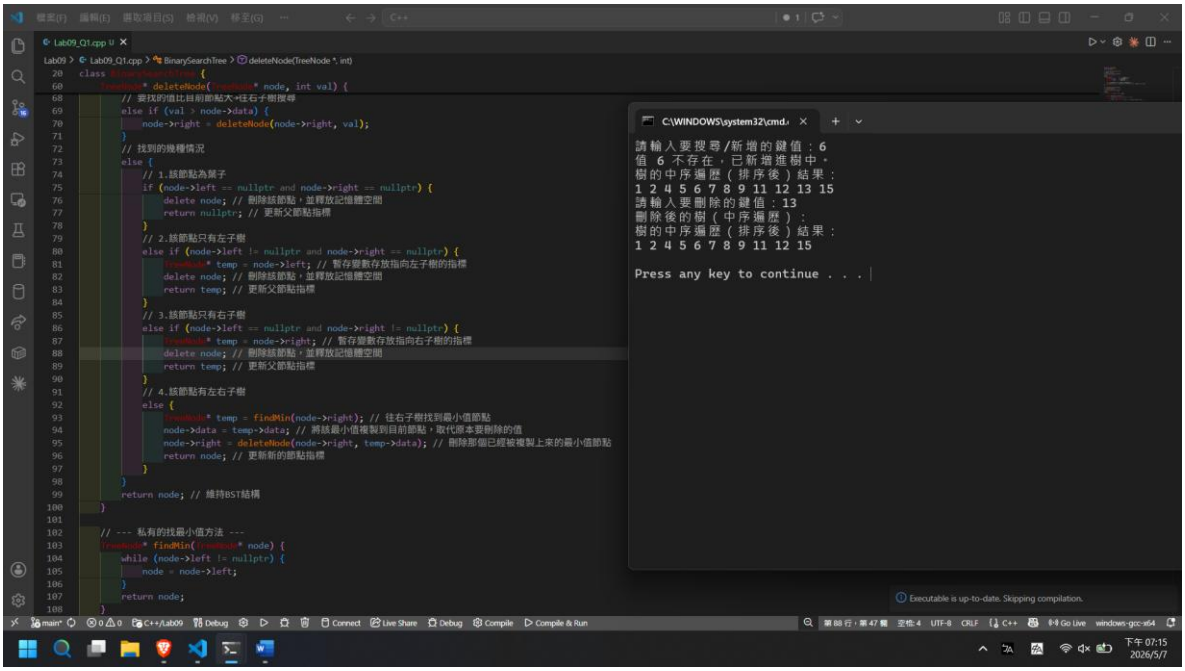
```
        // 找到的幾種情況
        else {
            // 1.該節點為葉子
            if (node->left == nullptr and node->right == nullptr) {
                delete node; // 刪除該節點，並釋放記憶體空間
                return nullptr; // 更新父節點指標
            }
            // 2.該節點只有左子樹
            else if (node->left != nullptr and node->right == nullptr) {
                TreeNode* temp = node->left; // 暫存變數存放指向左子樹的指標
                delete node; // 刪除該節點，並釋放記憶體空間
                return temp; // 更新父節點指標
            }
            // 3.該節點只有右子樹
            else if (node->left == nullptr and node->right != nullptr) {
                TreeNode* temp = node->right; // 暫存變數存放指向右子樹的指標
                delete node; // 刪除該節點，並釋放記憶體空間
                return temp; // 更新父節點指標
            }
            // 4.該節點有左右子樹
            else {
                TreeNode* temp = findMin(node->right); // 往右子樹找到最小值節點
                node->data = temp->data; // 將該最小值複製到目前節點，取代原本要刪除的值
                node->right = deleteNode(node->right, temp->data); // 刪除那個已經被複製上來的最小值節點
                return node; // 更新新的節點指標
            }
        }
        return node; // 維持 BST 結構
    }

    // --- 私有的找最小值方法 ---
    TreeNode* findMin(TreeNode* node) {
        while (node->left != nullptr) {
            node = node->left;
        }
        return node;
```

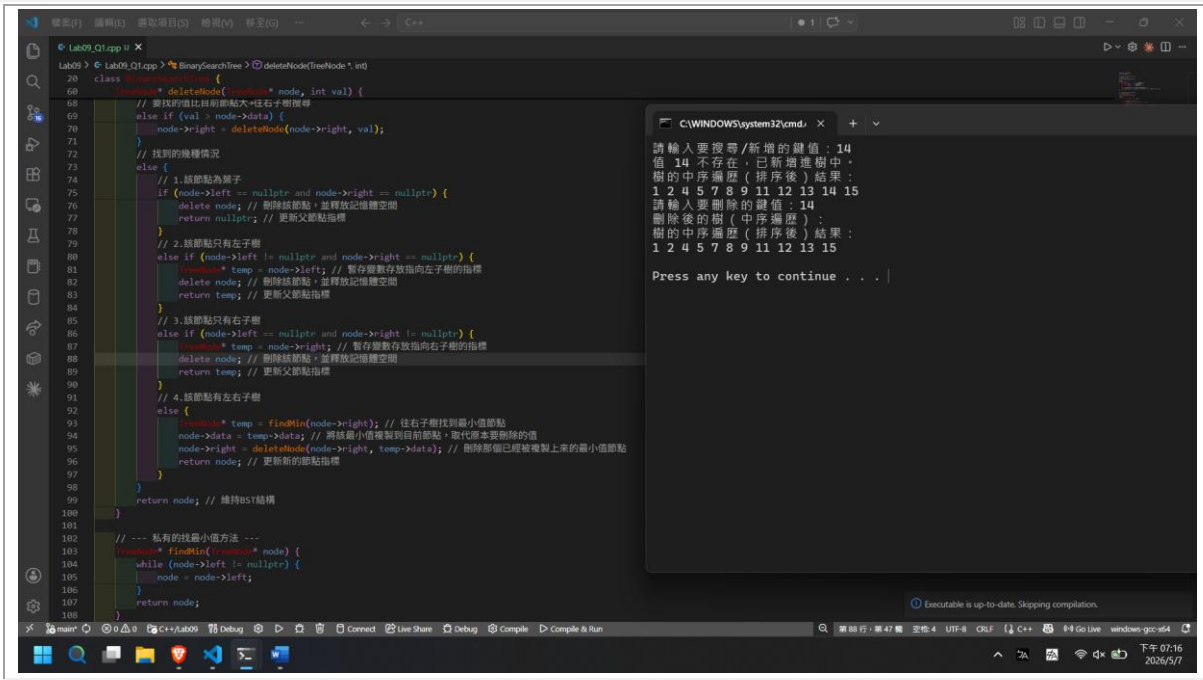
```
    }

public:
    // --- 建構子：初始化為空樹 ---
    BinarySearchTree() : root(nullptr) {}

    // --- 公開插入方法 ---
    void insert(int val) {
        root = insert(root, val);
    }

    // --- 公開搜尋方法 ---
    bool search(int target) {
        return search(root, target);
    }

    // --- 公開中序遍歷方法 ---
    void displayInOrder() {
        cout << "樹的中序遍歷（排序後）結果：" << endl;
        inOrderTraversal(root);
        cout << endl;
    }

    // --- 公開刪除方法 ---
    void deleteValue(int val) {
        root = deleteNode(root, val);
    }
};

int main() {
    BinarySearchTree bst; // 建立一棵空的樹

    // 預設插入的資料
    int values[] = { 7, 1, 4, 2, 8, 13, 12, 11, 15, 9, 5 };
    int n = sizeof(values) / sizeof(values[0]);

    // 把資料插入樹中
    for (int i = 0; i < n; ++i) {
```

```
        bst.insert(values[i]);
    }

    // 讓使用者輸入要插入的鍵值
    int key;
    cout << "請輸入要搜尋/新增的鍵值：";
    cin >> key;

    if (bst.search(key)) {
        cout << "值 " << key << " 已存在樹中，不新增。" << endl;
    }
    else {
        cout << "值 " << key << " 不存在，已新增進樹中。" << endl;
        bst.insert(key);
    }

    // 顯示目前樹的中序遍歷結果
    bst.displayInOrder();

    // 讓使用者輸入要刪除的鍵值
    int deleteKey;
    cout << "請輸入要刪除的鍵值：";
    cin >> deleteKey;

    // 刪除節點
    bst.deleteValue(deleteKey);

    // 顯示刪除後的樹
    cout << "刪除後的樹（中序遍歷）：" << endl;
    bst.displayInOrder();

    return 0;
}
```

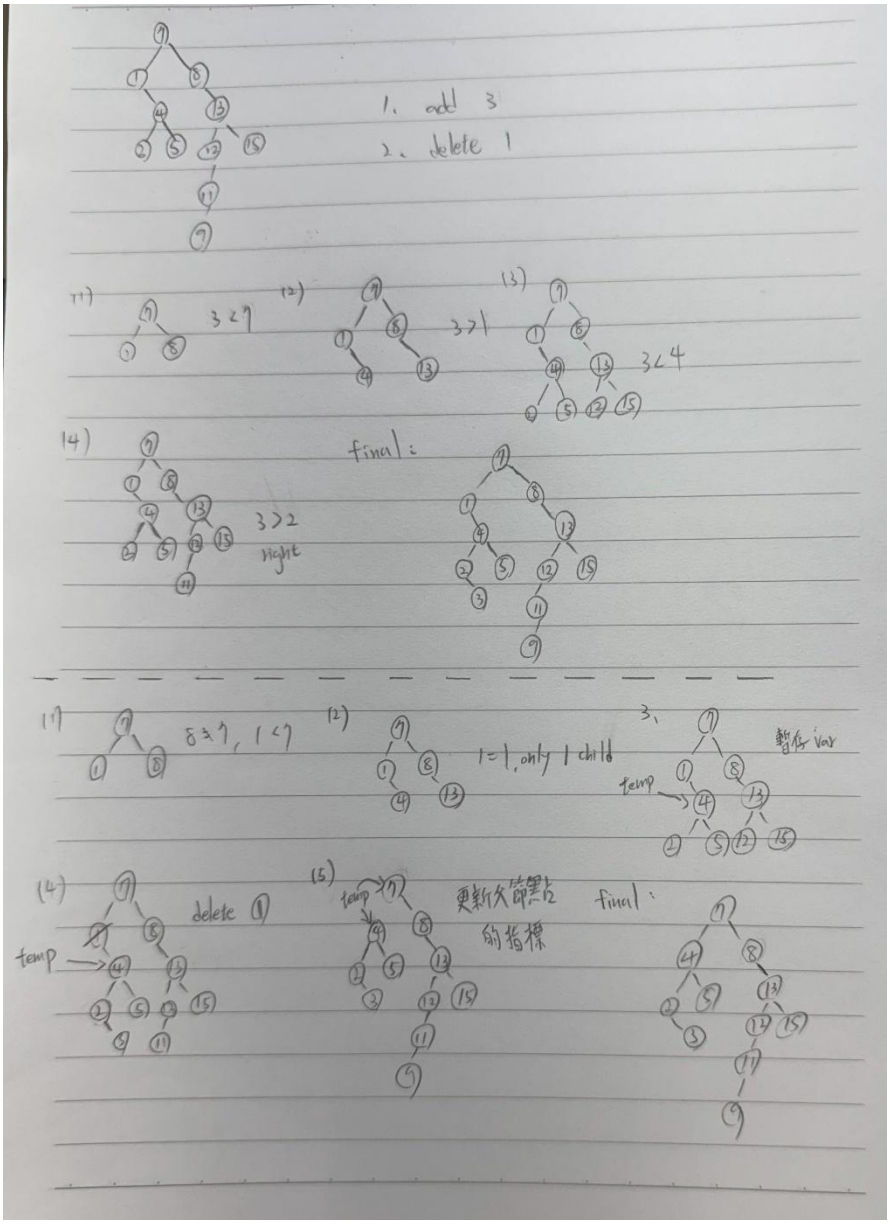
Result

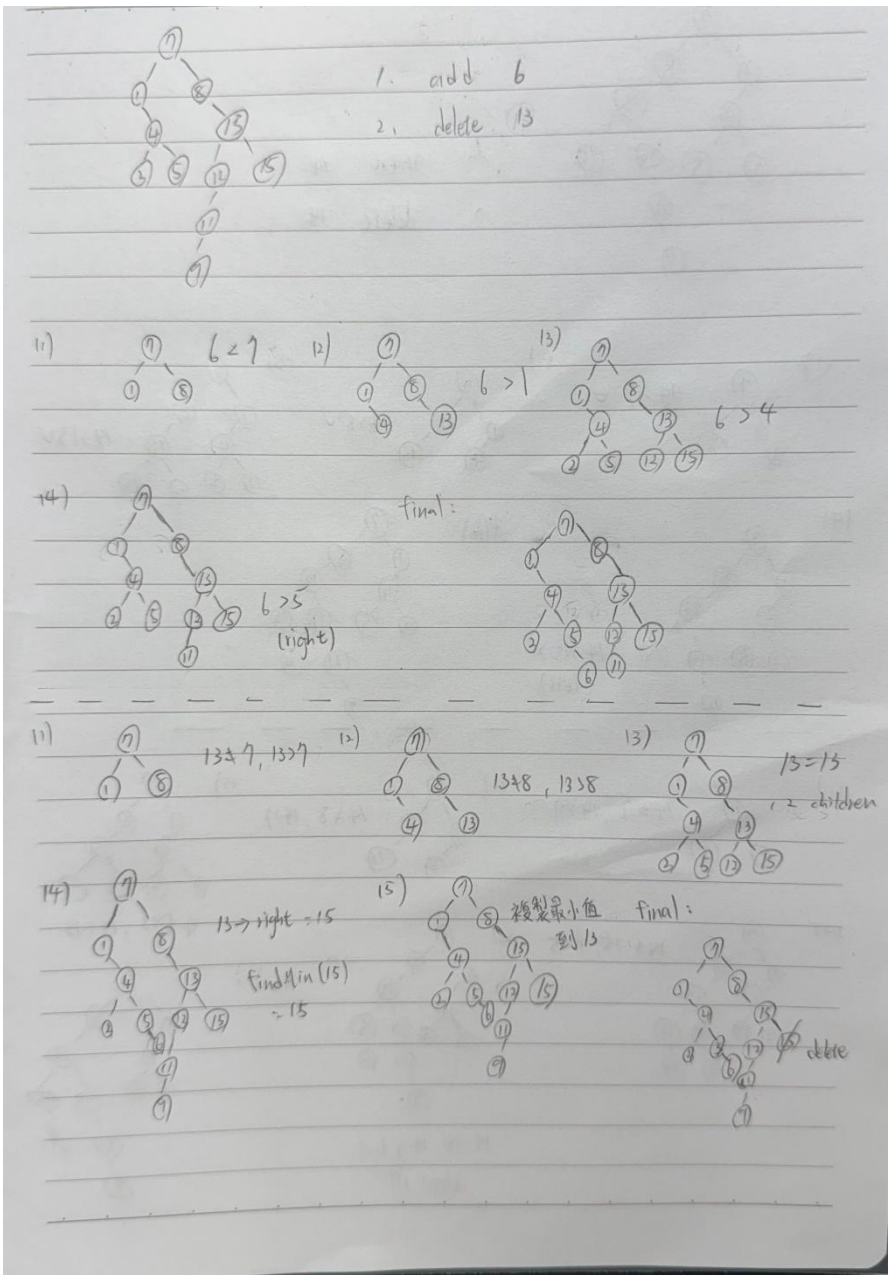

Lab09_Q1.cpp
Lab09 > Lab09_Q1.cpp > BinarySearchTree > deleteValue(int)
// 要找的值比目前節點大，往右子樹搜尋
else if (val > node->data) {
node->right = deleteNode(node->right, val);
// 找到的幾種情況
else {
// 1.該節點為葉子
if (node->left == nullptr and node->right == nullptr) {
delete node; // 刪除該節點，並釋放記憶體空間
return nullptr; // 更新父節點指標
// 2.該節點只有左子樹
else if (node->left != nullptr and node->right == nullptr) {
TreeNode* temp = node->left; // 暫存變數存放指向左子樹的指標
delete node; // 刪除該節點，並釋放記憶體空間
return temp; // 更新父節點指標
// 3.該節點只有右子樹
else if (node->left == nullptr and node->right != nullptr) {
TreeNode* temp = node->right; // 暫存變數存放指向右子樹的指標
delete node; // 刪除該節點，並釋放記憶體空間
return temp; // 更新父節點指標
// 4.該節點有左右子樹
else {
TreeNode* temp = findMin(node->right); // 往右子樹找到最小值節點
node->data = temp->data; // 將該最小值複製到目前節點，取代原本要刪除的值
node->right = deleteNode(node->right, temp->data); // 刪除那個已經被複製上來的最小值節點
return node; // 更新新的節點指標
return node; // 維持BST結構
// --- 私有的找最小值方法 ---
TreeNode* findMin(TreeNode* node) {
while (node->left != nullptr) {
node = node->left;
return node;
C:\WINDOWS\system32\cmd.
請輸入要搜尋/新增的鍵值：3
值 3 不存在，已新增進樹中。
樹的中序遍歷（排序後）結果：
1 2 3 4 5 7 8 9 11 12 13 15
請輸入要刪除的鍵值：1
刪除後的樹（中序遍歷）：
樹的中序遍歷（排序後）結果：
2 3 4 5 7 8 9 11 12 13 15
Press any key to continue . . .
Executable is up-to-date. Skipping compilation.

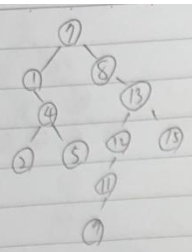

Lab09_Q1.cpp
Lab09 > Lab09_Q1.cpp > BinarySearchTree > deleteNode(TreeNode *, int)
// 要找的值比目前節點大，往右子樹搜尋
else if (val > node->data) {
node->right = deleteNode(node->right, val);
// 找到的幾種情況
else {
// 1.該節點為葉子
if (node->left == nullptr and node->right == nullptr) {
delete node; // 刪除該節點，並釋放記憶體空間
return nullptr; // 更新父節點指標
// 2.該節點只有左子樹
else if (node->left != nullptr and node->right == nullptr) {
TreeNode* temp = node->left; // 暫存變數存放指向左子樹的指標
delete node; // 刪除該節點，並釋放記憶體空間
return temp; // 更新父節點指標
// 3.該節點只有右子樹
else if (node->left == nullptr and node->right != nullptr) {
TreeNode* temp = node->right; // 暫存變數存放指向右子樹的指標
delete node; // 刪除該節點，並釋放記憶體空間
return temp; // 更新父節點指標
// 4.該節點有左右子樹
else {
TreeNode* temp = findMin(node->right); // 往右子樹找到最小值節點
node->data = temp->data; // 將該最小值複製到目前節點，取代原本要刪除的值
node->right = deleteNode(node->right, temp->data); // 刪除那個已經被複製上來的最小值節點
return node; // 更新新的節點指標
return node; // 維持BST結構
// --- 私有的找最小值方法 ---
TreeNode* findMin(TreeNode* node) {
while (node->left != nullptr) {
node = node->left;
return node;
C:\WINDOWS\system32\cmd.
請輸入要搜尋/新增的鍵值：6
值 6 不存在，已新增進樹中。
樹的中序遍歷（排序後）結果：
1 2 4 5 6 7 8 9 11 12 13 15
請輸入要刪除的鍵值：13
刪除後的樹（中序遍歷）：
樹的中序遍歷（排序後）結果：
1 2 4 5 6 7 8 9 11 12 15
Press any key to continue . . .
Executable is up-to-date. Skipping compilation.


Lab09_Q1.cpp
Lab09 > Lab09_Q1.cpp > BinarySearchTree > deleteNode(TreeNode *, int)
class BinarySearchTree {
TreeNode* deleteNode(TreeNode* node, int val) {
// 要找的值比目前節點大，往右子樹搜尋
else if (val > node->data) {
node->right = deleteNode(node->right, val);
}
// 找到的幾種情況
else {
// 1.該節點為葉子
if (node->left == nullptr and node->right == nullptr) {
delete node; // 刪除該節點，並釋放記憶體空間
return nullptr; // 更新父節點指標
}
// 2.該節點只有左子樹
else if (node->left != nullptr and node->right == nullptr) {
TreeNode* temp = node->left; // 暫存變數存放指向左子樹的指標
delete node; // 刪除該節點，並釋放記憶體空間
return temp; // 更新父節點指標
}
// 3.該節點只有右子樹
else if (node->left == nullptr and node->right != nullptr) {
TreeNode* temp = node->right; // 暫存變數存放指向右子樹的指標
delete node; // 刪除該節點，並釋放記憶體空間
return temp; // 更新父節點指標
}
// 4.該節點有左右子樹
else {
TreeNode* temp = findMin(node->right); // 往右子樹找到最小值節點
node->data = temp->data; // 將該最小值複製到目前節點，取代原本要刪除的值
node->right = deleteNode(node->right, temp->data); // 刪除那個已經被複製上來的最小值節點
return node; // 更新新的節點指標
}
}
return node; // 維持BST結構
}
// --- 私有的找最小值方法 ---
TreeNode* findMin(TreeNode* node) {
while (node->left != nullptr) {
node = node->left;
}
return node;
}
C:\WINDOWS\system32\cmd.
請輸入要搜尋/新增的鍵值：14
值 14 不存在，已新增進樹中。
樹的中序遍歷（排序後）結果：
1 2 4 5 7 8 9 11 12 13 14 15
請輸入要刪除的鍵值：14
刪除後的樹（中序遍歷）：
樹的中序遍歷（排序後）結果：
1 2 4 5 7 8 9 11 12 13 15
Press any key to continue . . .
Executable is up-to-date. Skipping compilation.

Discussion


1. add 3
2. delete 1
(1)
3<7
(2)
3>1
(3)
3<4
(4)
3>2
right
final:
(1)
8≠7, 1<7
(2)
1=1, only 1 child
3.
暫存 var
temp
(4)
delete 1
temp
(5)
temp
更新父節點
的指標
final:

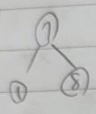

1. add 6
2. delete 13
11)
6<7
12)
6>1
13)
6>4
14)
6>5
(right)
final:
11)
13≠7, 13>7
12)
13≠8, 13>8
13)
13=13
, 2 children
14)
13→right =15
findMin(15)
=15
15)
複製最小值
到13
final:
delete



1. insert 14

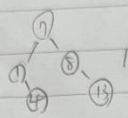
2. delete 14

(1)



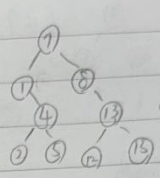
14>7 ✓

(2)



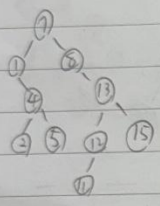
14>8 ✓

(3)



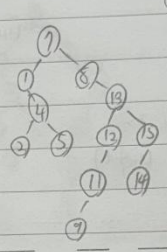
14>13 ✓

(4)



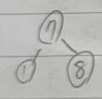
14>15 ✗
(left)

final:



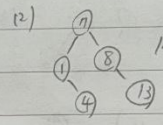
---

(1)



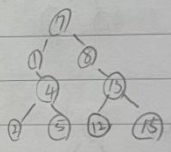
14≠7, 14>7

(2)



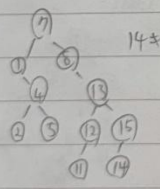
14≠8, 14>7

(3)



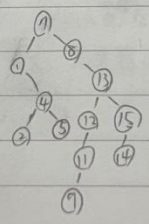
14≠13, 14>13

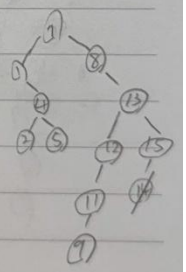
(4)



14≠15, 14<15

(5)



14=14, 14 is leaf
, delete 14

final: